

1.

```
/**
 * Reports the smallest integer in the given {@code Queue<Integer>}.
 *
 * @param q
 *         the queue of integer
 * @return the smallest integer in the given queue
 * @requires q != empty_string
 * @ensures <pre>
 *   min is in entries(q) and
 *   for all x: integer
 *     where (x is in entries(q))
 *     (min <= x)
 * </pre>
 */
private static int min(Queue<Integer> q) {
    int size = q.length();
    int min = q.front();
    for (int I = 0; I < size; i++){
        int current = q.dequeue();
        if( current < min){
            min = current;
        }
        q.enqueue(current);
    }
    Return min;
}
```

Q1: If we don't use the requires clause to specify that q must not be empty, we cannot use the dequeue and enqueue. They will cause errors. In addition, finding the minimum value in the queue is meaningless if it is empty.

Q2: The first line of the ensure clause ensures that the return number is an element of the queue. For example, the code forgets to set the smallest value as the first value of the queue and then the code outputs "0" or something else as the result. It cannot fulfill the request of the method.

2.

```
/**
 * Reports an array of two {@code int}s with the smallest and the
 * largest integer in the given {@code Queue<Integer>}.
 *
 * @param q
 *         the queue of integer
```

```

    * @return an array of two {@code int}s with the smallest and the
    *         largest integer in the given queue
    * @requires q != empty_string
    * @ensures <pre>
    * { minAndMax[0], minAndMax[1] } is subset of entries(q) and
    * for all x: integer
    *     where (x in in entries(q))
    *     (minAndMax[0] <= x <= minAndMax[1])
    * </pre>
*/
private static int[] minAndMax(Queue<Integer> q) {
    int size = q.length();
    int min = q.front();
    int max = q.front();
    for(int I = 0; I < size; i++){
        int current = q.dequeue();
        if(current < min){
            min = current;
        }
        if(current > max){
            max = current;
        }
        q.enqueue(current);
    }
    Int[] result = new int[2];
    Result[0] = min;
    Result[1] = max;

    Return result;
}

```

3.

```

private static int[] minAndMax(Queue<Integer> q) {
    int size = q.length();
    int min = q.dequeue();
    int max = min;
    q.enqueue(min);
    for (int I=1; I < size; I +=2){
        int a = q.dequeue();
        int b;
        if (I + 1 < size){
            b = q.dequeue();
        } else {
            B = a;

```

```

    }
    Int smaller = Math.min(a,b);
    Int larger = Manth.max(a,b);
    If (smaller < min){
        Min = smaller;
    }
    If(larger > max){
        Max = larger;
    }
    q.enqueue(a);
    if(i+1<size){
        q.enqueue(b);
    }
}
Int[] result = new int[2];
Result[0] = min;
Result[1] = max;

Return result;
}

```